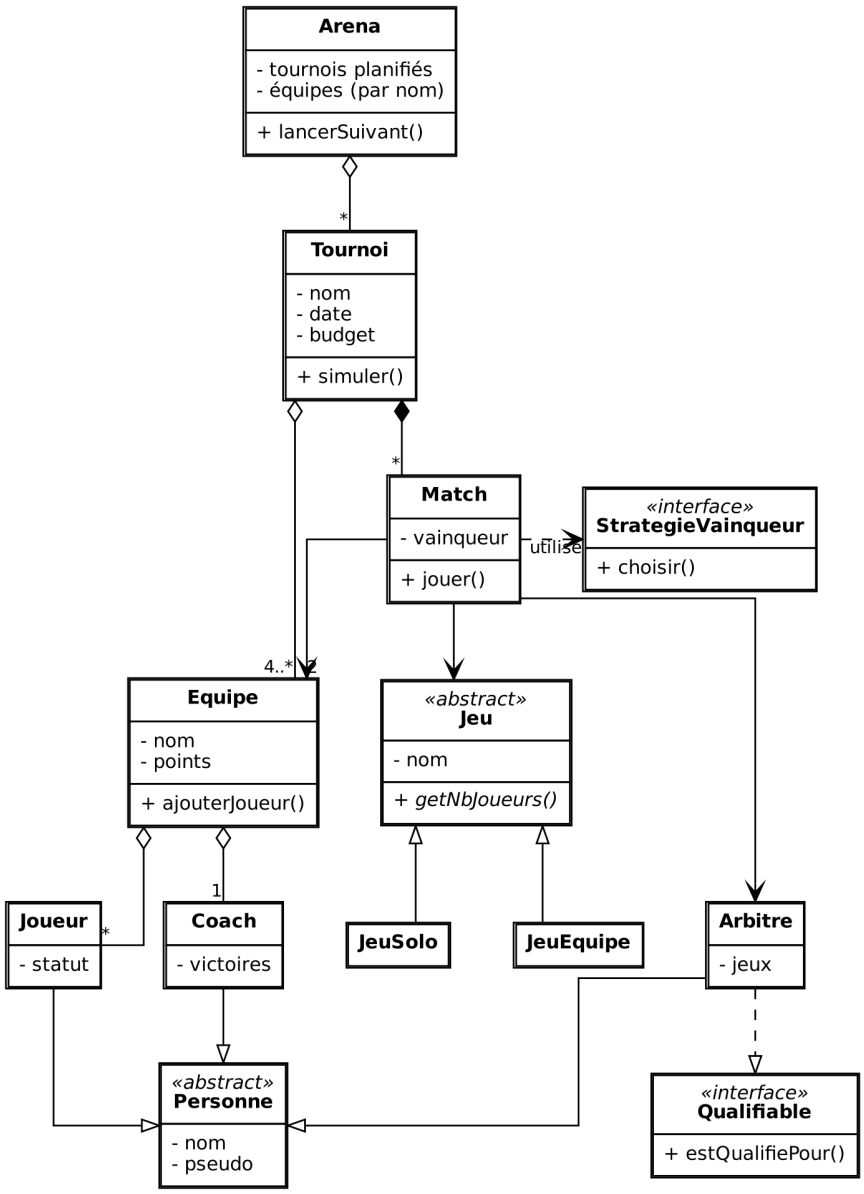
IMT Arena — Dossier de conception

TP Conception Orientée Objet — CI1 FISA ITR, IMT Nord-Europe — Java 17, Maven, JPA / H2, JUnit 5

1. Diagramme de classes (simplifié)



Triangle vide : héritage (trait plein) ou implémentation (pointillés). Losange vide : agrégation, losange plein : composition (les matchs n'existent que dans leur tournoi). Ne sont pas dessinés pour rester lisible : les enums, les 3 stratégies, `JeuFactory`, `MatchListener` / `JournalMatchs`, `RapportTournoi`, `RapportRepository` et `ComparateurBudget`.

2. Héritage, interfaces et encapsulation

- **Classes abstraites** : `Personne` regroupe ce que joueurs, coachs et arbitres ont en commun. `Jeu` a une méthode abstraite `getNbJoueurs()` que `JeuSolo` et `JeuEquipe` redéfinissent (polymorphisme) : un nouveau type de jeu = une nouvelle classe fille.
- **Interfaces** pour les capacités communes : `Qualifiable` (être qualifié pour un jeu) et `Comparable` (être classé).
- **Encapsulation** : attributs `private`, listes en lecture seule. Les objets vérifient les règles et lèvent une exception : joueur déjà dans une équipe, coach déjà pris, équipe contre elle-même, arbitre non qualifié, équipe trop petite ou inscrite deux fois, tournoi à moins de 4 équipes ou sans arbitre.

3. Collections choisies

Besoin	Collection	Pourquoi
Tournois planifiés, le plus proche d'abord	`PriorityQueue<Tournoi>`	`peek()` / `poll()` donnent toujours le plus proche. `Tournoi` est `Comparable` par date.
Classement des équipes, sans doublon, par points puis nom	`TreeSet<Equipe>`	Trié par le `compareTo` d'`Equipe`, pas de doublon. Recréé à chaque demande car les points changent.
Accès direct à une équipe par son nom	`HashMap<String, Equipe>`	Accès direct par la clé ; refuse deux équipes du même nom.
Jeux d'un arbitre	`HashSet<Jeu>`	Pas de doublon, `contains()` rapide, l'ordre ne compte pas.
Matchs d'un tournoi par équipe gagnante	`LinkedHashMap<Equipe, List<Match>>`	Regroupe les matchs par vainqueur en gardant l'ordre.
Tournois terminés, du plus gros au plus petit budget	`ArrayList` + `ComparateurBudget`	Autre ordre que l'ordre naturel (date), donc un `Comparator` à part.
Joueurs, équipes et matchs d'un tournoi	`ArrayList`	Peu d'éléments, ordre d'ajout, accès par index (joueur tiré au hasard en jeu solo).

4. Design patterns

- **Stratégie** — `StrategieVainqueur`, avec `TirageEquitable`, `FavoriseMieuxClasse` et `ToujoursMieuxClasse`. *Problème résolu :* le vainqueur se désigne de plusieurs façons ; on choisit la stratégie dans l'`Arena` sans modifier `Match`. `ToujoursMieuxClasse` donne un résultat connu pour les tests.
- **Observateur** — `MatchListener`, implémenté par `JournalMatchs`. *Problème résolu :* à chaque fin de match, le tournoi prévient tous les écouteurs sans les connaître ; on peut en ajouter sans modifier `Tournoi`.
- **Fabrique** — `JeuFactory.creer("Valorant")`. *Problème résolu :* créer un jeu à partir de son nom sans répéter son nombre de manches et de joueurs partout.

5. Simulation et persistance

- **Simulation** : `Arena.lancerSuivant()` prend le tournoi le plus proche dans la `PriorityQueue` et le joue en entier. Chaque match a 10 % de risque d'incident, sinon la stratégie choisit le vainqueur. Le `Random` a une graine fixe : le résultat est reproductible.
- **Fin automatique** : points des équipes, victoires des coachs, matchs arbitrés, passage pro des joueurs, vainqueur, réussite (au moins 70 % de matchs joués) et création du `RapportTournoi`.
- **Persistance** : seul `RapportTournoi` est une `@Entity` (id généré). Base H2 dans un fichier (`jdbc:h2:./data/arena`), configurée dans `persistence.xml`. `RapportRepository` sauvegarde le rapport à la fin de chaque tournoi et retrouve tous les rapports ou ceux d'un jeu (requête JPQL).
- **Synchronisation — non réalisée** : les tournois sont joués l'un après l'autre dans le programme principal, sans thread. Il n'y a donc pas d'`ExecutorService`, de durée de simulation ni de limite de 3 tournois en cours, et aucune synchronisation n'est nécessaire.